A Simple MNIST Digit Classifier Neural Network Made from scratch in Python

Thiruwaran Kalvin (T-Kalv)
Undergraduate Computer Science Student

August 12, 2025

1 Abstract

This research paper presents an implementation of a basic feed-forward neural network for classifying/recognising handwritten digits from the MNIST dataset, built from scratch in Python using only core libraries such as NumPy, Matplotlib, Pandas, and tqdm. This feed-forward neural network model, also known as a multilayer perceptron (MLP), achieves a classification probability accuracy of appoximately 88% for both the training and testing datasets. This demonstrates the viability of simple neural networks that recognise handwritten numerical digits without the need for external heavy libraries such as PyTorch or TensorFlow, instead utilising only core Python libraries and Juypter Notebook.

2 Introduction

Numerical Digit Classification is a specific field of Image Processing that focuses here on this paper, classifying and recognising handwritten low-level black and white numerical digit images using the MNIST Dataset[1]. This particular task of recognising handwritten digits is vital in the real-world as previously Bank Tellers played a crucial role in processing Cheques by manually verifying (looking) at the authenticity of the Cheques and verifying the Currency amount on the Cheques. This method was time-consuming and in-efficient due to manual Human validation as well as being prone to Human errors. As the level of technology/computation advanced, the need to accurately read hand-written numerical digits especially in sectors such as the Banking industry became more crucial to reduce the likelyhood of errors and reduce processing time. This results in Financial institutions diversting resources more effectivly to other services. Through the advancements in Artificial Intelligence and Machine learning, we can develop systems that can accuracy recognise hand-written digits much more efficient and effectively using hand-written datasets such as MNIST.

MNSIT (Modified Natual Institute of Standard and Technology) database is dataset that consits of handwritten numerical digits that is commonly used for image processing training and contains 60,000 training images used to train the Machine Learning Neural Network model and 10,000 testing images to validate the accuracy of the model. In this paper, we use this famous MNIST dataset which is the standard benchbark for Image Classification algorithms. As the MNIST database was constructed before the turn of the 21st Century, there have many implementations/existing solutions using the MNIST Dataset to classify and recognise hand-written numerical digits such as using tradition methods (including but not limited to KNN (K-Nearest Neighbours), SVM (Support Vector Machines)) and even Deep-Learning methods (including but not limited to CNN

(Convultional Neural Network), RNN (Recurrent Neural Network)). Arguably, one of the most famous implementations it the LeNet-5 architecture [3] which uses CNN consiting of 2 convolutions, 2 subsamplings and 3 fully connected layers with around 60000 trainable paramaters.

This paper proposes a light-weight simple MNIST Digit Classifier Neural Network [4] consiting of 2 layers with 784 neurons, built without the use of external heavy deep-learning libraries (only using libraries such as NumPy, Matplotlib) with a particular emphasis on understanding and manually implementing algorithms and functions. This work presents such algorithms and functions such as Forward Propagation, Backpropagation, ReLu (Rectified Linear unit) activation function, Softmax activation function. . . . This results in a reproducible implementation of simple MNSIT Digit Classifier Neural Network, with clear, hand-on demonstration of 'behind-the-scenes' of the network as well as the training procedure and performance benefits/outcomes.

This paper is organised as follows: Section 3 discusses related work, followed by Section 4 which outlines the Methodology and architecture, Section 5 presents Experimental Evaluation and finally Section 6 concludes with findings and future work.

3 Related Work

Before working on this 'Simple MNIST Digit Classifier Neural Network', previously a 'Simple Neural Network' was developed from scratch in Python for XOR [4]. This previous project implemented key concepts such as Forward Propagation, Backpropagation, Mean-Squared Error Loss and the Sigmoid Activation function using only core libraries such as NumPy and Matplotlib. The sucess of XOR having a probability of appoximately 95% helped to inspire the next challenge of handling a more real-world dataset MNIST, implementing these key concepts on this 'Simple MNSIT Digit Classifier Neural Network'.

Previous research into using the MNIST dataset for classifying and recognising handwritten digits has produced a wide-range of models with a variety of complexity and performance benefits to the traditional manual, human-validation of handwritten digits. LeNet-5 [3], introduced in 1998 having achived appoximately a 1%[5] test error probability which resulted in appoximatly 99% probability accuracy with the MNSIT dataset, implemented using CNN (Convultional Neural Networks). While LeNET-5[5], achived a respectable, high probability accuracy using CNN, Max Pooling with 2 convultions, 2 subsampling, 3 fully connected layers and around 60000 trainable paramaters, this paper investigates the performance of a simple multilayer perceptron (MLP) using a feed-forward neutal network trained from scratch without the need of heavy, high-level frameworks. However, more modern implementations do use these heavy, high-level frameworks such as PyTorch [6], Keras [7], TensorFlow [8] utilising the performance benefits of GPUs (Graphics Processing Unit), NPU (Neural Processing Unit), TPUs (Tensor Processing Units) which help to optimise the speed of learning/training as well with the use of advanced optimisers (including but not limited to Adam). Here due to the added cost of utilising such performance enhancers, the system here presented in this paper will instead utilise modern CPUs for computation and training.

4 Methodology

4.1 Dataset

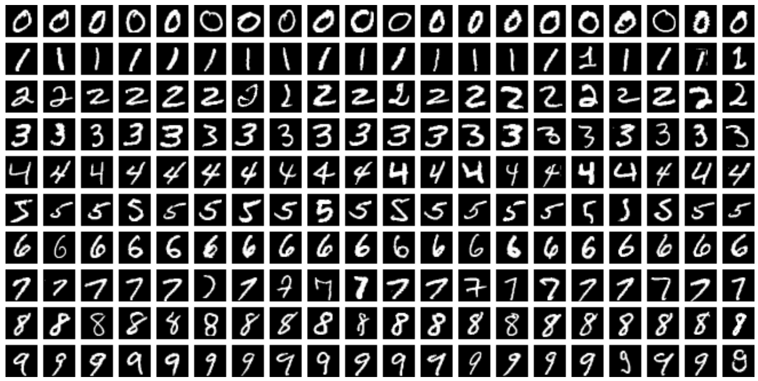

Figure 1: Sample handwritten digits from the MNIST dataset. Image by Suvanjanprasai, CC BY-SA 4.0.

The MNIST dataset [9] contains gray-scale, black and white images of hand-drawn numerical digits from zero to nine (0, 1, 2, 3, 4, 5, 6, 7, 8, 9). Each image is 28 pixels in height by 28 pixels in width) contains a total of 784 pixels, where each pixel value is an integer from 0 to 255 resulting in black to white. The dataset consits of 60000 training images for training the model and 10000 testing images for testing the trained model. The training dataset in this paper has 785 columns which are the 'label'. This is the corresponding '0 to 9' digit that was handwritten and the remaining columns are the '0 to 255' pixel values associated with the black and white image. The testing dataset is the same as the training dataset execept without containing the labels to validate the accuracy of the model.

4.2 Network Architecture

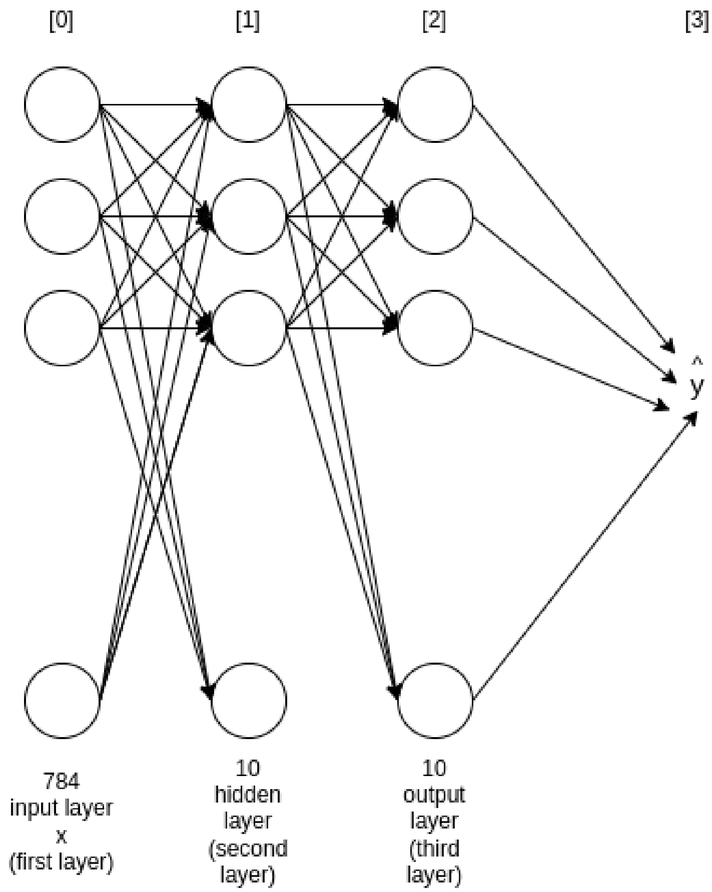

Figure 2: Simplified Neural Network Architecture used in this Project

The Neural Network [Figure 2] is a fully-connected, feed-forward neural network that consits of an input layer, hidden layer and output layer. The input layer (first layer x) consits of 784 neurons which correspons to the 784 pixels in the 28x28 flattened image for each of the images in the MNSIT dataset. The hidden layer (second layer) consits of 10 neurons where the corresponding weights and biases connecting to this layer and initialised with random values between -0.5 to 0.5. In addition, the activiation function used in the hidden layer is the ReLu (Rectified Linear unit) activation function. The output layer (third layer) consits of 10 neurons which is the precition of what numerical digit the input image represents from 0 to 9 resulting in 10 possible numerical digit classes.

The parameters of this Neural Network consits of a 'weight1' which has a shape of (10,784) connecting the input to the hidden layer, 'bias 1' which has a shape of (10,1) which are the biases for the hidden layer, 'weight 2' which has a shape (10,10) connecting the hidden layer to the output layer and 'bias2' which has a shape of (10,1) which are the biases of the output layer. This results in this simple MNIST Digit Classifier Neural Network having a total of 7960 trainable parameters with 7840 weights, 10 biases between the input layer and hidden layer, 100 weights, 10 biases between the hidden layer and putput layer.

4.3 Forward Propagation

1: Forward Propagation Pseudocode Algorithm

```
FUNCTION forwardPropagation(weight1, bias1, weight2, bias2, X):
    Z1 = matrixMultiply(weight1, X) + bias1
    A1 = ReLuFunc(Z1)
    Z2 = matrixMultiply(weight2, A1) + bias2
    A2 = softMaxFunc(Z2)
    RETURN Z1, A1, Z2, A2
```

2: ReLu Activation Function Pseudocode

```
FUNCTION ReLUFunc(Z):
    RETURN MAXIMUM(0, Z)
```

3: Softmax Activation Function Pseudocode

```
FUNCTION softmaxFunc(Z):
    maxZ = MAX(Z, axis=0)
    shiftedZ = Z - maxZ
    expZ = EXP(shiftedZ)
    sumExpZ = SUM(expZ, axis=0)
    RETURN (expZ / sumExpZ)
```

The Forward Propagation algorithm used in this project, takes the input data image and runs through the Neural Network layers to determine what the output is going to be. Here in this 'Simple MNIST Digit Classifier Neural Network' the input data is a flattened 28x28 pixel image consiting of 784 pixels. This input image is then represented as a matrix which each row is an example with 784 columns corresponding to 1 pixel in the image. This matrix is then transposed resulting in the rows examples becoming columns examples.

In the first layer (unactivated layer), the input 'X' is matrix multiplied by the weight1 (first weight) and then the bias term is added, resulting in Z1 which is the first unactivated first layer in the matrix (as illustrated in '1: Forward Propagation Pseudocode Algotithm' above).

The second layer is a linear combination of the first layer which results in a purely linear transformation, hindering the Neural Network's ability to model non-linear relationships. This leads to an application of a ReLu (Rectified Linear unit) activation function which is applied to Z1 to introduce non-linearities where it processes each element in Z returning the value if it greater than zero or return zero otherwise (as illustrated in '2: ReLU Activation Function Pseudocode' above) leading to A1.

In the pre-activation, output layer, A1 is then matrix multiplied by weight2 (second weight) and then the second bias term is added, resulting in Z2. This leads to an application of the Softmax activation function (as illustrated in '3: Softmax Activation Function Pseudocode' above) which is used to convert the raw values into probability values for each of the 10 possible numerical digit classes (0, 1, 2, 3, 4, 5, 6, 7, 8, 9), resulting in A2.

4.4 Backpropagation

The Backpropagation algorithm used in this project, uses the predicted output probabilities from the Forward Propagation algorithm as a starting point. The Backpropagation algorithm then computes the gradients of the loss function with respect to each of the parameters in the Neural Network. It does this by calculating how much the Forward Propagation prediction deviated by and compares it to the actual label from the MNIST dataset resulting in an error loss value and calculates how much the previous weights and biases contributed and adjust these values, from the output layer moving backward through the network in the opposite direction to the movement of Forward Propagation algorithm.

$$\text{one\_hot}(i, K) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \quad \text{where the 1 appears at index } i \text{ and } K \text{ is the total number of classes.}$$

Figure 3: One-Hot Encoding Algorithm

Firstly, a One-Hot Encoding algorithm is applied (as illustrated in 'Figure 3: One-Hot Encoding Algotithm') where it creates a matrx array of zeros with 10 output classes, 0 to 9, and goes through each row in the matrix specified by the label Y and sets it to 1. Then it tranposes the matrix so that each coloumn is an example instead of a row.

After the One-Hot Encoding Matrix Y is returned, the output layer calculation is performed. This is done by first subtracting the One-Hot Encoding Matrix Y labels from the predicted probabilities from the Forward Propagation algorithm. This reults in dZ2 which is error of the second layer in the Nerual Network, which represents the deviation from the correct label.

dW2, which is the weights gradient in the second layer, is calculated by first calculating the size of the One-Hot Encoding Matrix Y, m. Then it multiplies dZ2 with the transpose of A1 (which is the hidden layer activations) and divided by m. This results in the derivative of the loss function with respect to the weights in layer 2.

dB2, which is the bias gradient for the second layer in the Neural Network, is computed by getting the size of the One-Hot Encoding Matrix Y, m and then then summing all of the elements in dZ2 and dividing by m. This results in average of the absolute error for the seccond layer, dB2.

After the average of the absolute error for the second layer is obtained, the error is then back propagated to the hidden layer by multiplying the transpose of weight2 with dZ2 and then multipying by the ReLu derivative activation function to Z1. This results in dZ1 which represents how much the first hidden layer has been deviated by.

Then DW1, which is the first layer weight gradient, is computed by getting the size of the One-Hot Encoding Matrix Y, m and then multiplying dZ1 with the input Matrix X that has been transposed and divide by m. This results in the how much weight1 in the first layer contributed to next layer's (hidden layer) error.

Finally, dB1, which is the first layer biases gradient, is computed by getting the size of the One-Hot Encoding Matrix Y, m and then summing all the elements in dZ1 and dividing by m. This results in how much bias1 contributed to the first layer to the Neural Network overall error.

4.5 Training

4: Initialise Paramaters Pseudocode

```
FUNCTION initialiseParameters():
    weight1 = randomMatrix(10, 784) - 0.5
    weight2 = randomMatrix(10, 10) - 0.5
    bias1 = randomMatrix(10, 1) - 0.5
    bias2 = randomMatrix(10, 1) - 0.5
    RETURN weight1, weight2, bias1, bias2
```

5: Update Paramaters Pseudocode

```
FUNCTION updateParameters():
    weight1 = weight1 - α x dW1
    weight2 = weight2 - α x dW2
    bias1 = bias1 - α x dB1
    bias2 = bias2 - α x dB2
    RETURN weight1, weight2, bias1, bias2
```

In order for this simple MNIST Digit Classifier Neural Network to learn from the training dataset, a gradientDescent function is used where it iteratively adjusts the Neural Networks parameters (weights and biases) to accurately predict numerical handwritten digits based on errors made before.

Firstly, we assign random starting values to weight1, weight2, bias1, bias2 in each of the layers and subtract by 0.5 which helps to ensure the initial weights and biases are centered around zero than being positive (as illustrated in '4: Initialise Paramaters Pseudocode' above). This reults in the Neurons in the Neural Network start with different parameters while training and ensures the Network learns effectively.

After the paramters have been initialised, forwardPropagation is performed where it takes the input data image and runs through the Neural Network layers to determine what the output data is going to be (as mentioned earlier in this paper in 'Section 4.3 - Forward Propagation').

The predicted ouput probabilities from forwardPropagation is then used in Backpropagation, where it computes the gradients of the loss function with respect to each of the parameters in the Neural Network and calulated the deviation of its predictions from the correct label from the MNIST dataset (as mentioned earlier in this paper in 'Section 4.4 - Backpropagation').

After the gradients of the loss function with respect to each of the parameters in the Neural Network have been computed, updateParameters is performed where each weight1, weight2, bias1, bias2 are adjusted depending on the results in the gradientDescent algorithm. Here in updateParameters, each parameter is updated by getting the product of the rate of learning α and subtracting it with its corresponding gradient. This process is repeated for each of the paramaters where it gradually reduces the error loss by moving the paramaters in the direction where it improved the performance of the Neural Network.

Finally, here in this simple MNIST Digit Classifier Neural Network Model, for every 10th iteration we fetchPrecitions and fetchAccuracy while training.

5 Experimental Evaluation

5.1 Accuracy Results

5.2 Observations

5.3 Limitations and Anomalies

6 Conclusion and Future Work

Appendix

References